

A Real-Time 2D Virtual Patient for Motivational Interviewing Training

Multimodal Architecture, SQL-Governed Prompting, and LoRA-Driven Viseme Personalization

Thesis manuscript (synthesized from project research materials)

April 29, 2026

Abstract

Motivational interviewing (MI) is effective but difficult to practice at scale because human standardized-patient resources are limited and expensive. This work presents a browser-deployable virtual patient system that closes a speech loop from user microphone input through automatic speech recognition, large-language-model dialogue guided by MI-oriented prompts, text-to-speech with temporal alignment, and two-dimensional mouth animation driven by Rhubarb-style viseme cues. The platform merges administrative and educational workflows (authentication, cases, assignments, transcripts) with a low-cost 2D avatar representation that avoids photograph-based avatars in the primary practice mode. Optional diffusion-based pipelines (local, Hugging Face, or Gemini backends) generate identity-consistent viseme image sets, with an optional Stable Diffusion 1.5 LoRA hook for improved articulation and identity lock. We summarize the end-to-end architecture, implementation anchors, limitations, and a roadmap for empirical studies on skill transfer, perceived empathy, and engagement. Full project documentation is reproduced in the appendices via included source files.

Introduction

Motivational interviewing is a collaborative, person-centered counseling style that strengthens intrinsic motivation for change (Miller & Rollnick, 2012). Training clinicians to use MI well requires repeated practice with realistic interpersonal feedback. Conventional approaches rely on role-play with peers, standardized patients, or supervisors. While high in fidelity, these methods scale poorly across cohorts and schedules.

Interactive simulation and virtual standardized patients have long been proposed to complement human training; recent systematic reviews in health professions education emphasize outcome measurement, fidelity, and instructional design when evaluating such systems. Advances in automatic speech recognition (ASR), generative language models, and neural text-to-speech (TTS) make continuous, voice-first conversation technically feasible in a web browser. However, many educational chat systems remain text-centric, and many avatar prototypes prioritize entertainment visuals over behaviorally grounded clinical pedagogy.

This manuscript synthesizes the design rationale, system architecture, and technical documentation of the **Virtual Patient — Unified Local** codebase: a merged application combining an earlier React-based platform (cases, admin, practice, assignments, transcripts, image library, global prompt,

voice chat) with a two-dimensional mouth-shape pipeline inspired by a prior proof-of-concept using Rhubarb lip-sync, without requiring a user-uploaded photo avatar in the primary practice flow.

Problem Statement and Contributions

Clinical training problem

Learners need **safe, repeatable** conversation practice that preserves the rhythm and prosody of real dialogue. Text-only interfaces under-embodiment the interaction; purely passive video lacks the adaptive back-and-forth that MI emphasizes.

Research gaps

Three gaps motivate the present work:

1. **MI systems are often text-centric**, with limited embodied feedback during rehearsal.
2. **Avatar systems are often visual demonstrations** rather than integrated training stacks with scenario governance, transcripts, and assignment workflows.
3. **Few low-cost 2D pipelines** unify MI-specific prompt composition, a real-time speech loop, and **personalized** viseme assets suitable for educational deployment.

Contributions

This work contributes an **integrated, modular** educational stack:

- **SQL-governed prompt composition** merging a global MI policy with per-case system prompts for reproducibility and controlled interventions.
- A **closed speech loop**: browser capture → Whisper transcription → dialogue generation → ElevenLabs TTS with alignment → **mouth cues** consumed by the frontend.
- **Personalized 2D viseme generation** (optional) via text-to-image and image-to-image diffusion, with documented **LoRA** dataset conventions for local identity-consistent mouth packs.
- A **research-ready web application** (auth, dashboards, live practice) suitable for studies on communication training at scale.

System Overview

End-to-end runtime pipeline

The runtime path can be summarized as follows:

1. The learner speaks into the browser microphone; audio is encoded and sent over a WebSocket.
2. The backend transcribes audio using OpenAI Whisper (**whisper-1** by default).
3. The backend composes the final system behavior policy from persisted **global** and **case-level** prompts, then obtains a model response (streaming).
4. Spoken patient output is synthesized with ElevenLabs; alignment metadata is transformed into a time-ordered **cue** sequence (Rhubarb **mouthCues** semantics).
5. The frontend queues audio chunks, plays them sequentially, and drives the **2D avatar** by swapping PNG mouth shapes (**A-H, X, blink**) according to cue timestamps.

WebSocket events coordinate turns so that perceived latency remains manageable while preserving transcript traceability for downstream assessment.

Repository lineage

The **unified-local** project intentionally preserves sibling folders for earlier proofs of concept (**virtual-patient-poc-1**, **virtual-patient-poc-2d**) while integrating their capabilities into a single deployable tree. Prerequisites include Python 3.11+, Node.js 18+, **ffmpeg**, the Rhubarb Lip Sync binary (path configurable), and API credentials for cloud services where used.

Methodology and Subsystems

Prompt orchestration via SQL

A global behavioral prompt is stored as an application setting (**global_prompt**). Each training case stores a case-level system prompt. Session logic **merges** global and case prompts to form the effective policy. Faculty can update one global MI guideline consistently across cohorts while preserving scenario-specific nuance—supporting **experimental reproducibility** when prompt wording is an independent variable.

Speech-to-text

Whisper-based transcription treats STT as a first-class stage in the turn loop rather than an optional accessory. The transcript is shown in the user interface and fed forward to dialogue generation, enabling both voice-first interaction and text-based review.

Text-to-speech and alignment

ElevenLabs synthesis returns audio together with **alignment** information. The backend maps alignment to discrete mouth states for lip-sync. Configuration exposes voice parameters such as **stability** (prosodic consistency), **similarity_boost** (adherence to a reference voice identity), and **style** (expressive coloration). These parameters are positioned for controlled studies on perceived empathy, consistency, and realism; Appendix C reproduces concrete preset values for calm, anxious, defensive, and other affective regimes from the project documentation.

Lip synchronization and 2D rendering

Separation of concerns is explicit: the backend determines cue timing; the frontend renders frames. **audio_chunk** messages carry encoded audio plus the cue array. The React component **PatientAvatar2D** resolves a base URL for viseme PNGs (default static assets or per-case generated folders), selects the active shape including a **blink** overlay with guards to avoid replacing wide-open speech visemes during playback, and applies optional **head** transforms (yaw **rotateY**, pitch **rotateX**, roll **rotate**) and **eye** offsets via **object-position**. The current baseline uses static head motion (zeros) for stability; the transform interface is already present for future prosody-driven motion models.

Optional viseme personalization (diffusion + LoRA)

The viseme pipeline supports three backends (**local**, **hf**, **gemini**) selected by **VISEME_IMAGE_BACKEND**. A common flow generates a base face **A.png** (text-to-image), then derives remaining shapes (B–H,

X, blink) via image-to-image passes with **identity-lock** prompts. Local generation can load optional LoRA weights via CUSTOM_LOCAL_LORA_DIR and CUSTOM_LOCAL_LORA_SCALE. Dataset layout, caption schema, and collection heuristics are specified in Appendix B.

User Interface and Educational Workflow

The unified frontend provides registration and login backed by JWT authentication, role-based dashboards (administrator, student), case editing (including viseme reference upload or prompt-driven base-face generation), assignments, and a **live practice session** pairing transcript, voice controls, and the animated 2D patient. This yields a **browser-only** rehearsal workflow appropriate for institutions that cannot rely on specialized client installations.

Limitations and Future Work

Known limitations include: static head motion in the current baseline; opportunity to expand per-case viseme URL wiring in all runtime paths; and the need for **clinical validation** and learner-outcome studies beyond engineering feasibility.

Recommended empirical directions:

- Compare MI skill gains against a **text-only** baseline.
- Manipulate TTS slider settings and measure **perceived empathy** and trust.
- Contrast **personalized** viseme packs against generic packs on engagement and subjective realism.

Conclusion

The unified virtual patient platform addresses a practical bottleneck in MI education by combining scalable voice dialogue with low-cost 2D articulation and administratively complete training workflows. Its modular architecture allows independent substitution of ASR, dialogue models, TTS providers, cue generators, and renderers—supporting both deployment and controlled research.

References

Miller, W. R., & Rollnick, S. (2012). *Motivational interviewing: Helping people change* (3rd ed.). Guilford Press.

Appendix A: Virtual Patient — Unified Local (README)

Virtual Patient - Unified Local

Merged application: ****POC-1**** feature set (React/MUI, cases, admin, practice, assignments, tra

The original folders `virtual-patient-poc-1` and `virtual-patient-poc-2d` are unchanged; this p

Prerequisites

- Python 3.11+ recommended
- Node.js 18+ for the frontend
- [ffmpeg](https://ffmpeg.org/) (used by pydub before Rhubarb)
- [Rhubarb Lip Sync](https://github.com/DanielSWolf/rhubarb-lip-sync) - the default binary path

Setup

```
```bash
cd virtual-patient-unified-local

Backend
cd backend
cp ../.env.example ../.env
Edit ../.env - set OPENAI_API_KEY, ELEVEN_LABS_API_KEY, JWT_SECRET, optional SEED_ADMIN_*

python3 -m pip install -r requirements.txt
python3 main.py
API: http://127.0.0.1:5001 health: GET /health
```
```

In a second terminal:

```
```bash
cd virtual-patient-unified-local/frontend-react
npm install
npm run dev
Opens Vite (default port from vite.config.ts, often http://localhost:3000)
```
```

Leave `VITE\_API\_URL` unset so the browser talks to the same origin and Vite ****proxies**** `/api`

Auth (local)

- Register and login use the API (`POST /api/auth/register`, `POST /api/auth/login`) and store
- Optional: set `SEED\_ADMIN\_EMAIL` and `SEED\_ADMIN\_PASSWORD` in `.env` to create an admin user

2D avatar assets

Mouth PNGs live in `frontend-react/public/static/shapes/` (copied from POC-2D). The practice p

Viseme generation from `A.png` (optional, admin)

The viseme pipeline supports three backends controlled by `VISEME\_IMAGE\_BACKEND`:

1. `local` (default): fully local diffusion on your machine, no cloud credits.
2. `hf`: Hugging Face router/providers (requires token permissions and available credits).
3. `gemini`: Google Gemini image generation via `google-genai` SDK (set `GEMINI\_API\_KEY` in `.

All backends use the same generation flow:

- **A.png** from text (`txt2img`)
- **B-H, X, blink** from `A.png` (`img2img`) with strict identity-lock prompts

For the local backend, install backend dependencies (`torch`, `diffusers`, etc.) from `backend`.

The custom local generator includes automatic quality checks (identity/background/mouth-shape)

For HF backend, create a token at [Hugging Face access tokens](https://huggingface.co/settings,

Per case (recommended): In **Admin -> Edit case**, either **Upload viseme reference (A.png)**

Global (demo only): `POST /api/admin/visemes/generate-from-reference-a` with JSON `{ "prompt

Per-case API: `POST /api/admin/visemes/generate-for-case` with `{ "case_id": "...", "prompt

`A.png` is written when you use **Generate base face from prompt**. Lower `LOCAL_IMG2IMG_STRENGTH

Optional LoRA hook for local generation:

- Set `CUSTOM\_LOCAL\_LORA\_DIR` and `CUSTOM\_LOCAL\_LORA\_SCALE` in `.env`
- Dataset/training format reference: `backend/docs/viseme\_lora\_dataset.md`

API notes

- WebSocket `audio\_chunk` messages include `cues` (Rhubarb `mouthCues`) for the 2D face.
- SQLite database file defaults to `backend/aimii.db` when `DATABASE\_URL` is unset.

Appendix B: Viseme LoRA Dataset Specification

Viseme LoRA Dataset Spec

This project supports optional LoRA loading for local viseme generation via:

- `CUSTOM\_LOCAL\_LORA\_DIR`
- `CUSTOM\_LOCAL\_LORA\_SCALE`

Use this dataset format when preparing a LoRA that improves identity lock + mouth articulation

Directory layout

...

```
viseme_lora_dataset/  
  images/  
    avatar_A_0001.png  
    avatar_B_0001.png  
    avatar_C_0001.png  
    ...  
  metadata.jsonl  
...
```

`metadata.jsonl` schema

Each line:

```json

```
{"file_name": "images/avatar_F_0001.png", "text": "solo portrait, same character, viseme F, jaw d
```

```

Required fields:

- `file\_name`: relative path to image
- `text`: training caption
- `viseme`: one of `A,B,C,D,E,F,G,H,X,blink`
- `identity\_id`: character identity label

Data collection rules

- Keep camera/framing/background fixed across visemes.
- Include at least 30-50 examples per viseme for one identity before training.
- Prefer 512x512 PNG images.
- Ensure mouth shape is clearly visible and exaggerated enough for `F/G/H`.
- Include `X` and `blink` examples; these stabilize idle and eye-closure behavior.

Prompt template recommendation

Use consistent prompt scaffolding:

- `same character`
- `same pose`
- `same camera framing`
- `same lighting/colors`
- `white background`
- `viseme <key> mouth shape`

Integration hook

At runtime, set:

```
```env
VISEME_IMAGE_BACKEND=local
CUSTOM_LOCAL_LORA_DIR=/absolute/path/to/lora/weights
CUSTOM_LOCAL_LORA_SCALE=0.8
```
```

The local generator automatically attempts to load/fuse LoRA weights for txt2img and img2img p

Appendix C: ElevenLabs Mood and Slider Examples

ElevenLabs 3-Mood Slider Examples

These are three practical presets for the virtual patient voice, each paired with a generated I

Mood Presets

1. Calm / Cooperative
 - stability: 0.78
 - similarity_boost: 0.82
 - style: 0.20
 - sample: `docs/presentation/audio/mood\_calm.mp3`
2. Anxious / Worried
 - stability: 0.36
 - similarity_boost: 0.75
 - style: 0.68
 - sample: `docs/presentation/audio/mood\_anxious.mp3`
3. Defensive / Irritable
 - stability: 0.46
 - similarity_boost: 0.72
 - style: 0.74
 - sample: `docs/presentation/audio/mood\_defensive.mp3`

Notes

- Keep the spoken text constant across moods for clean comparisons.
- If delivery sounds too flat, increase `style` by 0.05 to 0.10.
- If voice identity drifts, increase `similarity\_boost` by 0.05.

Contrast Set (Subtle -> Extreme)

These use the same sentence as the presets above, but maximize expressive contrast for demos.

1. Subtle
 - stability: 0.82
 - similarity_boost: 0.84
 - style: 0.12
 - sample: `docs/presentation/audio/mood\_subtle.mp3`
2. Medium
 - stability: 0.55
 - similarity_boost: 0.78
 - style: 0.48
 - sample: `docs/presentation/audio/mood\_medium.mp3`

- 3. Extreme
 - stability: 0.24
 - similarity_boost: 0.68
 - style: 0.90
 - sample: `docs/presentation/audio/mood\_extreme.mp3`

Emotion Set (Happy / Sad / Angry)

- 1. Happy
 - stability: 0.58
 - similarity_boost: 0.80
 - style: 0.72
 - sample: `docs/presentation/audio/mood\_happy.mp3`
- 2. Sad
 - stability: 0.74
 - similarity_boost: 0.78
 - style: 0.30
 - sample: `docs/presentation/audio/mood\_sad.mp3`
- 3. Angry
 - stability: 0.28
 - similarity_boost: 0.70
 - style: 0.88
 - sample: `docs/presentation/audio/mood\_angry.mp3`

Emotion-Matched Sentences (Happy / Sad / Angry)

Each mood uses a different line that semantically matches the intended emotion.

- 1. Happy
 - sentence: "That is wonderful news, and I am really glad we are making progress together to
 - stability: 0.56
 - similarity_boost: 0.80
 - style: 0.74
 - sample: `docs/presentation/audio/mood\_happy\_sentence.mp3`
- 2. Sad
 - sentence: "I feel overwhelmed and tired, and it has been hard for me to stay hopeful late
 - stability: 0.76
 - similarity_boost: 0.78
 - style: 0.28
 - sample: `docs/presentation/audio/mood\_sad\_sentence.mp3`
- 3. Angry
 - sentence: "I am frustrated because no one seems to listen, and I am tired of repeating my
 - stability: 0.27

- similarity_boost: 0.70
- style: 0.90
- sample: `docs/presentation/audio/mood\_angry\_sentence.mp3`

Expressive Emotion Set (Higher Contrast)

Generated with `model\_id: eleven\_multilingual\_v2` and stronger punctuation/prosody cues.

1. Happy (Expressive)

- sentence: "This is amazing news! I feel so much lighter, and I am genuinely excited about
- stability: 0.38
- similarity_boost: 0.78
- style: 0.95
- sample: `docs/presentation/audio/mood\_happy\_expressive.mp3`

2. Sad (Expressive)

- sentence: "I don't know... I feel exhausted, and honestly, it has been hard to keep going
- stability: 0.86
- similarity_boost: 0.80
- style: 0.08
- sample: `docs/presentation/audio/mood\_sad\_expressive.mp3`

3. Angry (Expressive)

- sentence: "No. I'm frustrated. I keep explaining this, and nobody is listening to me!"
- stability: 0.16
- similarity_boost: 0.68
- style: 1.00
- sample: `docs/presentation/audio/mood\_angry\_expressive.mp3`